Final Report:
Orbital

Cryptocurrency Market Data & News Aggregation Platform: Distributed Systems Course
2025–2026

Aldo Visconti
aldo.visconti@studio.unibo.it

November 2, 2026

Orbital is a distributed, microservices-based platform that aggregates real-time cryptocurrency market data and financial news behind a single API gateway, with a Kotlin Multiplatform client (desktop and web) on top. Application logic is decomposed into two specialized services, **market-service** and **news-service**, fronted by a **gateway** that proxies, caches, and rate-limits every client request.

The project focuses on keeping that aggregated data available and durable independently of any single component failing. The **CAP theorem** trade-off is explicitly addressed: the system prioritizes **availability** over strict consistency, since a client is always better served a slightly stale cached price than an error. **Fault tolerance** is implemented through real, verified infrastructure: both PostgreSQL databases run with a streaming-replication standby, and the Redis cache runs under a three-node Sentinel quorum, so a single-instance failure does not take the gateway's caching or rate-limiting down with it. Market data is continuously ingested by a standalone worker, decoupled from API traffic, which persists an append-only time series and performs a one-time historical backfill for newly observed coins.

Disclaimer

During the preparation of this work, the author used Claude Code (Anthropic) as a pair-programming and writing assistant: for implementing and debugging backend services, designing and drafting the Postgres replication / Redis Sentinel / distributed rate-limiting infrastructure, performing a security and code-quality audit of the resulting system, and drafting portions of this report from the resulting codebase and the author's own direction.

After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of this report and of the underlying project.

Contents

1 Concept **3**

2 Requirements Elicitation and Analysis **4**
2.1 Functional Requirements 4
2.2 Non-functional Requirements 5
2.3 Implementation Choices 6
2.4 Relevant Distributed System Features 7

3 Design **8**
3.1 Architecture 8
3.2 Infrastructure 9
3.3 Modelling 10
3.3.1 Domain Entities 10
3.3.2 The Backfill Gate 10
3.4 Interaction 11
3.5 Behaviour 13
3.6 Data and Consistency Issues 13
3.6.1 CAP Theorem Position 13
3.6.2 Consistency Model 14
3.6.3 Shared Data and Synchronization 14
3.7 Fault-Tolerance 14
3.7.1 Service Isolation via Independent Databases 14
3.7.2 Replication 15
3.7.3 Fail-Open Cache and Rate Limiter 15
3.7.4 Idempotent Backfill on Restart 16
3.7.5 Database Connection Retry and Startup Ordering 16
3.8 Availability 16
3.8.1 Caching 16
3.8.2 Load Balancing 16
3.8.3 Network Partition Behaviour 17
3.9 Security 17
3.9.1 Current State 17
3.9.2 Proposed Identity Design (Not Implemented) 18
3.9.3 Proposed Authorization Model (Not Implemented) 18
3.9.4 Keycloak's Own Fault Tolerance (Not Implemented) 18

4 Implementation **20**
4.1 Network Protocols and Data Representation 20
4.2 Data Persistence and Querying 20
4.3 Authentication and Authorization 20
4.4 Frontend Implementation 20
4.5 Containerization and Orchestration 21

4.6 Technological Details 21

5 Validation **22**
5.1 Automatic Testing 22
5.1.1 Test Infrastructure 22
5.1.2 Test Coverage 22
5.1.3 How to Run 23
5.2 Acceptance Testing 23

6 Deployment **24**
6.1 Prerequisites 24
6.2 Installation from Scratch 24
6.2.1 Docker Compose Services 25
6.2.2 Startup Ordering and Health Checks 26
6.3 Rebuilding Individual Services 26
6.4 Schema Management 26

7 User Guide **26**

8 Self-evaluation **27**
8.1 Aldo Visconti 27

9 Future works **28**

1 Concept

Orbital is a set of web services (a gateway and two backend services) fronting a shared persistence/caching layer, paired with a Kotlin Multiplatform client application (desktop and web) that consumes them.

Use case. The use case is a personal market-data and news dashboard: a user wants to see current cryptocurrency prices, a coin's price history, and recent news, without visiting several separate sites.

- **What:** The software continuously ingests cryptocurrency prices and news, caches and rate-limits access to that data at a single gateway, and presents it through a searchable, filterable client.
- **Roles:** There is currently a single, anonymous role, since there are no user accounts and nothing distinguishes one caller from another beyond their IP address for rate-limiting purposes (see section 3.9 for a proposed design that would introduce real roles).
- **Interaction:** Users interact via a desktop window or a web browser. Interaction is pull-based and relatively low-frequency from any single user's perspective (opening

the app, searching for a coin, expanding its chart, switching a news category), but the client also auto-refreshes periodically, and many independent users may be doing this concurrently.

- **Data Storage:** The system does not store any user-specific data, since there is no concept of a user account to attach data to. The data it *does* store, price snapshots and news articles, is global, shared across every user, persisted in two separate PostgreSQL databases (one per backend service).

Why is distribution needed? Distribution was deliberately chosen for the following reasons, not imposed artificially:

- **Resource Sharing:** All clients observe the same market and news data, so it makes sense to fetch and store it once, centrally, rather than have every client poll CoinGecko independently, which would also multiply load against CoinGecko's own rate limits [2].
- **Fault Tolerance:** A single unreplicated database or cache instance would make the whole platform's persistence path a single point of failure; section 3.7 describes how this is addressed.
- **Independent Scalability:** The gateway is the one component every client talks to, so it is the one component that needs to be horizontally scalable independently of the two backend services behind it; section 3.8 describes how this was designed for.
- **Educational Purpose:** To implement and analyze real distributed-systems patterns, namely streaming database replication, quorum-based cache failover, a distributed rate limiter, and CAP-theorem trade-offs, in a project small enough to reason about end to end.

2 Requirements Elicitation and Analysis

2.1 Functional Requirements

FR1 The system shall provide current market data (price, market cap, 24h volume, 24h price change) for the top N cryptocurrencies by market capitalization.
Acceptance: `GET /api/v1/market/prices` returns a JSON array of at least N coins, each carrying all of the above fields.

FR2 The system shall provide historical price data for an individual cryptocurrency, built from the system's own continuously-ingested observations rather than only the upstream API's short-range sample.
Acceptance: `GET /api/v1/market/history/{coinId}` returns a chronologically ordered series of prices for a coin that has previously been observed by the system.

FR3 The system shall provide a list of news articles, optionally filtered by category.
Acceptance: `GET /api/v1/news[?category=]` returns articles consistent with the requested category filter, or all categories if none is given.

FR4 The system shall provide read access to its own previously-stored news archive, independent of the live upstream feed's availability.
Acceptance: `GET /api/v1/news/history[?category=]` returns previously persisted articles even when the upstream news source is unreachable.

FR5 The client application shall allow a user to search and filter cryptocurrencies by free text, and news articles by category.
Acceptance: typing into the market search field narrows the displayed coin list; selecting a news category narrows the displayed articles to that category.

FR6 The client application shall present an individual coin's historical price trend visually.
Acceptance: expanding a coin in the client renders a chart built from FR2's data, falling back to the live snapshot's short-range sample if no history has been ingested yet.

FR7 The system shall continuously and automatically ingest market data on a fixed interval, independent of API request traffic, and shall perform a one-time historical backfill for each newly observed coin.
Acceptance: price snapshots keep accumulating in storage over time even with zero API traffic; a coin observed for the first time has a long history available immediately rather than only from the moment it was first observed.

2.2 Non-functional Requirements

NFR1 (Availability) The gateway shall keep serving already-cached responses even if a backend service, the cache, or the rate limiter's backing store is temporarily unavailable, rather than failing the request outright.
Acceptance: a Redis outage must not cause the gateway to reject requests outright (verified in this project by the rate limiter's fail-open behaviour, section 3.7).

NFR2 (Fault Tolerance) Loss of a single PostgreSQL primary shall not lose previously committed data, given a configured streaming-replication standby.
Acceptance: `pg_stat_replication` on a primary shows an actively streaming replica with a matching log position (verified manually in this project, section 5.2).

NFR3 (Scalability) The gateway shall be horizontally scalable to multiple replicas without each replica enforcing an independent, and therefore effectively multiplied, rate limit.
Acceptance: a shared, Redis-backed counter enforces one limit per client across every replica (implemented and unit-tested; full multi-replica behaviour under load is discussed as not yet verified in section 9).

NFR4 (Performance) Market and news read endpoints requested repeatedly within a short window shall be served from cache rather than re-fetching upstream on every request.
Acceptance: two identical requests issued within the cache TTL result in exactly one upstream call (covered by an automated test, section 5.1).

NFR5 (Observability) All backend services shall expose health and metrics endpoints, scraped and visualized centrally.
Acceptance: `/health` and `/metrics` respond on the gateway and both backend services; a Prometheus/Grafana dashboard is provisioned automatically with no manual setup.

2.3 Implementation Choices

1. **Technical Stack:** The entire codebase, backend, client, and shared library modules, is written in Kotlin, with Ktor [5] as the backend HTTP server/client framework.
   - *Justification:* Ktor's coroutine-based concurrency model suits the I/O-bound proxy/aggregation workload of a gateway and two backend services well, and keeping the whole backend on one language and build toolchain removes a language boundary that would otherwise need to be crossed for no architectural reason.
2. **Client Platform:** Kotlin Multiplatform with Compose Multiplatform, targeting both JVM desktop and `wasmJs` web from a single codebase.
   - *Justification:* avoids maintaining two separate UI codebases for what is otherwise the same client logic, and keeps the client in the same language as the backend.
3. **Database:** PostgreSQL 15 [9], accessed through the Exposed ORM, with one independent database per backend service (`market_data`, `news_data`).
   - *Justification:* a mature, well-understood relational engine with first-class streaming replication, directly relevant to NFR2. Separate databases per service keep each service's persistence failure domain isolated from the other's.
4. **Cache and Coordination:** Redis, accessed via Jedis, behind either a plain `JedisPool` or a Sentinel-aware `JedisSentinelPool` [7] depending on configuration.
   - *Justification:* Redis is the de facto standard for this kind of short-TTL proxy cache, and Sentinel is its own built-in high-availability mechanism rather than a third-party add-on, keeping the operational surface small.
5. **Load Balancing:** nginx, in front of however many gateway replicas are running.
   - *Justification:* a minimal, well-understood reverse proxy was sufficient; nothing in this project's requirements called for a full service mesh.

6. **Deployment:** Docker Compose [3] orchestrating the entire stack on a single host.
    - *Justification:* reproducibility across development and this report's own manual verification work (section 5.2), without the operational overhead of a multi-host orchestrator that this project's scale does not need.

2.4 Relevant Distributed System Features

Transparency. Real, and central to the design. `orbital-app` only ever talks to the gateway (section 3.1); it has no knowledge of there being two backend services, two PostgreSQL primaries, a Redis Sentinel quorum, or (potentially) several gateway replicas behind a load balancer. Location and replication are meant to be invisible to it by construction.

Fault tolerance, dependability. Highly relevant, and the area with the most concrete work in this project: PostgreSQL streaming replication for both databases, a Redis Sentinel quorum, retry-with-backoff on startup, and fail-open behaviour in both the cache and the rate limiter. Uninterrupted service is not strictly required (this is not a payments system), but losing previously-collected price/news history to a single container crash was judged unacceptable given FR2/FR4's whole point is to keep that history around. Section 3.7 details how.

Scalability. Relevant specifically at the gateway, the one component every client and every request passes through; not particularly relevant at the two backend services, which are not designed to need more than one replica each. Section 3.8 covers the design; section 9 is explicit about what has and has not actually been exercised under load.

Security, trust. The weakest feature in the current implementation (see section 3.9): there are no user accounts, so technically there is no *sensitive* data to protect yet, and no authentication or authorization exists. This is acceptable for a public read-only aggregator, but it is a real gap relative to any future version of the product that introduces accounts, which is why section 3.9 proposes a concrete design rather than leaving the question unaddressed.

Resource sharing. Central to the system's existence (section 1): price and news data is fetched once and shared across every client, rather than each client fetching independently; the Redis cache, the Redis-backed rate-limit counters, and both PostgreSQL databases are all resources shared across however many gateway/service replicas are running.

Openness, interoperability. Relevant at the boundaries: the backend talks to CoinGecko's public REST/JSON API as an external, independently-versioned system,

and the client and backend communicate over plain HTTP/JSON rather than a project-specific binary protocol, so either side could in principle be replaced independently as long as that contract holds.

Evolvability, maintainability. Actively designed for: the backend is split into independently buildable Gradle modules, dependency versions are centralized, and every push runs automated tests, static analysis (detekt), and formatting checks (ktfmt/spotless) in CI, with releases cut automatically from Conventional Commit history. This matters because the project has already gone through several rounds of adding infrastructure (replication, Sentinel, a distributed rate limiter) without a rewrite.

Performance, concurrency. Relevant but not under strict quantitative targets: there is no SLA on response time. Ktor's coroutine-based I/O and the proxy cache (section 3.8) keep the gateway from doing redundant work, which matters more for not wasting CoinGecko's own rate-limited quota than for raw latency.

Economy, costs. A real, concrete driver of at least one implementation decision already made: news-service currently serves a mocked fallback feed rather than paying for a NewsAPI subscription, a cost-based trade-off made explicitly rather than left implicit.

3 Design

This chapter explains the strategies used to meet the requirements identified in section 2, motivated against the features identified in section 2.4.

3.1 Architecture

Orbital uses a **Microservices** architecture decomposed along functional boundaries. After analysis, three macro concerns were identified:

- **Market Data Logic:** Performed by `market-service` and the standalone `market-ingestor` worker. Handles CoinGecko integration, price persistence, and historical backfill. Exposes a REST API for reads; the worker itself exposes nothing and only writes.
- **News Logic:** Performed by `news-service`. Handles the (currently mocked) news feed and its own persisted archive. Exposes a REST API.
- **Edge:** Handled by `gateway` (the single client-facing entry point, caching and rate limiting) and `gateway-lb` (the load balancer in front of however many gateway replicas are running).

This **functional decomposition** ensures that market-service and news-service, two independent external integrations with independent failure modes, can evolve, be deployed, and fail independently of one another, and that the one component actually

needing to scale (the gateway) is not coupled to the two that do not. The client, `orbital-app`, is architecturally a classic thin client: all domain logic and persistence live behind the gateway, and the client's own role is presentation and local UI state only.

3.2 Infrastructure

The infrastructure is composed of the following decoupled components, all orchestrated by one `docker-compose.yml`:

- **Client-side:** `orbital-app`, a Compose Multiplatform application (desktop JVM and `wasmJs` web). It communicates with the gateway exclusively via REST/HTTP.
- **Edge Layer:** `gateway-lb`, an nginx instance acting as a reverse proxy. It provides a single entry point on port `8080`, hides however many `gateway` replicas sit behind it, and re-resolves the `gateway` service name on every request rather than caching it once, so scaling is actually reflected.
- **Service Layer:** `gateway` (routing, caching, rate limiting), `market-service` and `news-service` (independent Ktor applications, each owning its own database), and `market-ingestor` (a worker process with no HTTP interface).
- **Persistence Layer:** Two independent PostgreSQL 15 primaries: `postgres-market` (owned by market-service/market-ingestor) and `postgres-news` (owned by news-service), each with its own streaming-replication standby.
- **Cache / Coordination Layer:** A Redis primary and replica, monitored by a three-node Sentinel quorum.
- **Monitoring:** Prometheus (scraping all three HTTP services) and Grafana.
- **Administration:** pgAdmin, for inspecting both PostgreSQL instances.

All of these currently run on a single host, on one Docker bridge network (`orbital-net`); nothing in the design assumes same-host placement, but nothing in the current deployment spans more than one host either. That would be the natural next step if Orbital needed genuine geographic distribution, which section 1 does not identify as a driver for this product.

Components find each other by **name**, not by hardcoded IP address: Docker Compose's embedded DNS resolves each service's name (e.g. `postgres-market`, `redis`, `redis-sentinel-1`) to whichever container is currently running it. For Redis specifically, this is deliberately *not* enough on its own: a client that resolved `redis` once and cached the connection would keep talking to the old master after a Sentinel failover. Both the gateway and news-service therefore talk to Redis through the Sentinel quorum (`REDIS_SENTINEL_HOSTS`) rather than a fixed hostname, asking the quorum who the current master is on every new connection. This makes the infrastructure **location-transparent** in the sense relevant to section 2.4: nothing downstream of the gateway needs to know which physical Redis node is currently serving as master.

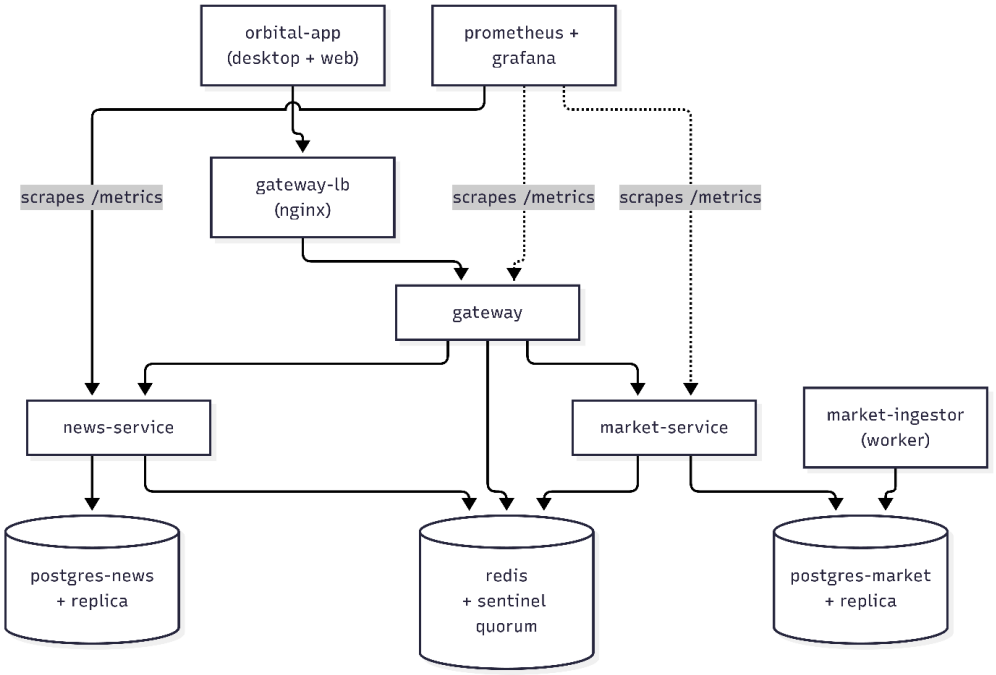


Figure 1: Infrastructure topology: client, edge, service, and persistence/cache layers.

3.3 Modelling

3.3.1 Domain Entities

The domain is partitioned between the two backend services, each owning its database outright:

- **Price Snapshot (market-service):** coin id, symbol, name, current price, market cap, 24h volume, 24h change, and an observation timestamp. One row per observation.
- **News Article (news-service):** id, title, description, URL, source, category, and a published timestamp. One row per observation.

Both tables map to exactly one infrastructural location each: price snapshots live only in `postgres-market`'s `market_prices` table, news articles only in `postgres-news`'s `news` table. There is no replication of domain data *between* the two backend services, only within each one's own PostgreSQL primary/standby pair.

3.3.2 The Backfill Gate

A single idempotency check, `MarketRepository.hasHistory(coinId)`, is the mechanism that makes FR7's one-time historical backfill safe to re-run unconditionally on

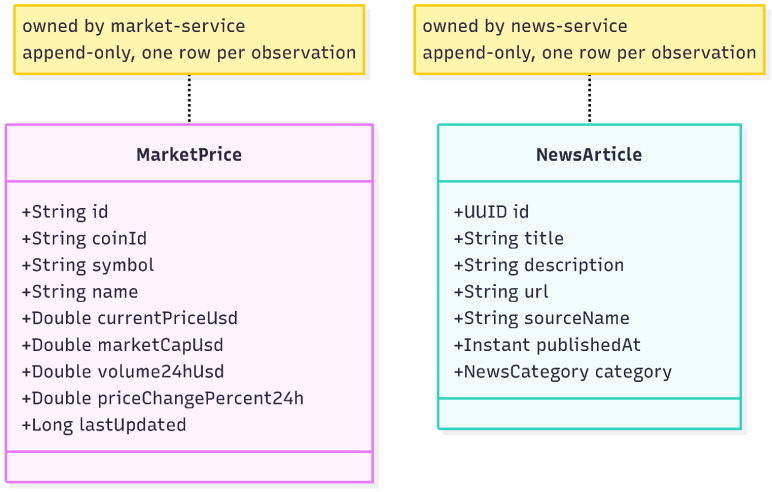


Figure 2: The two domain entities, each owned by one backend service.

every startup of `market-ingestor`, rather than needing its own separate completion-tracking table. Before fetching a coin's full `market_chart` history, the worker checks whether any row already exists for that coin; if one does, the backfill is skipped entirely, with no CoinGecko call and no delay. This single boolean gate is what lets the worker's startup sequence be simply "for every top coin, backfill if needed, then enter the polling loop" without the worker needing to remember, across restarts, which coins it has already handled: that memory already exists, as a side effect, in the very table the backfill writes to. This same idempotency is what makes the worker's restart behaviour (section 3.7) safe, since a crash mid-backfill simply means the next startup re-checks the gate and continues from wherever the data actually stopped.

There is no equivalent gate on the regular polling loop, by design: each polling tick is expected to append a fresh row regardless of what came before, since FR7's whole point is a continuously growing observation log, not a single current-value row to be kept idempotent against repeats.

3.4 Interaction

All interaction in Orbital is synchronous **request/response** over HTTP. The client (`orbital-app`) issues a request to the gateway; the gateway either answers directly from its Redis/in-process cache or proxies the request onward to the relevant backend service, which in turn either answers from its own database or makes an outbound HTTP call to CoinGecko (or the mocked news source) before persisting the result and responding. There is exactly one interaction pattern in use throughout, client-server request/reply, chosen deliberately over, for instance, a publish/subscribe or streaming pattern, since nothing in section 2 calls for push updates: the client's auto-refresh (FR5/FR6) is itself just a periodic pull, not a server-initiated push.

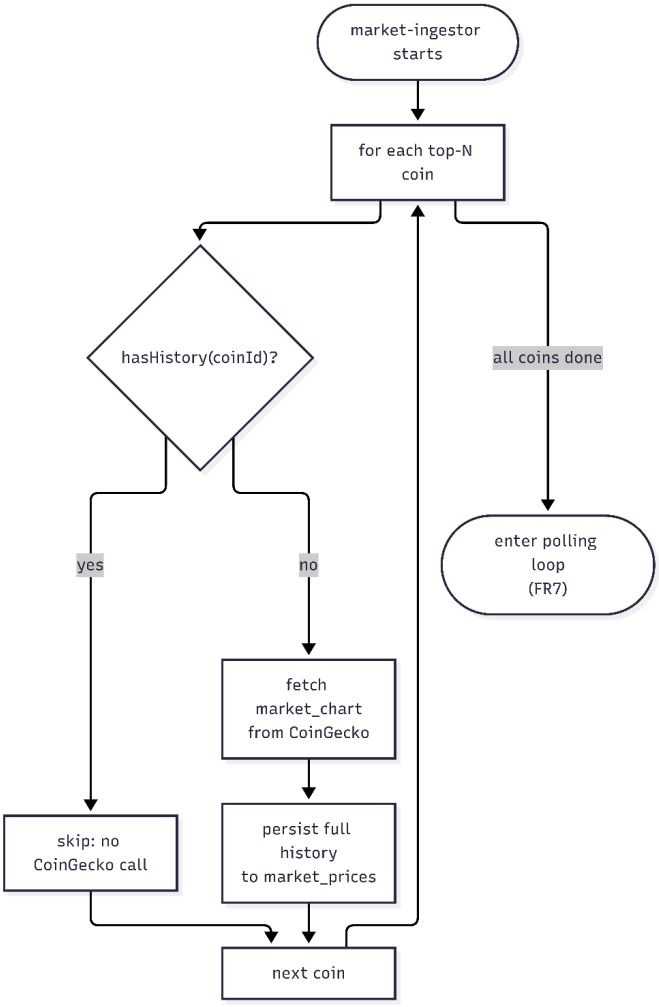


Figure 3: The backfill gate's decision logic on worker startup.

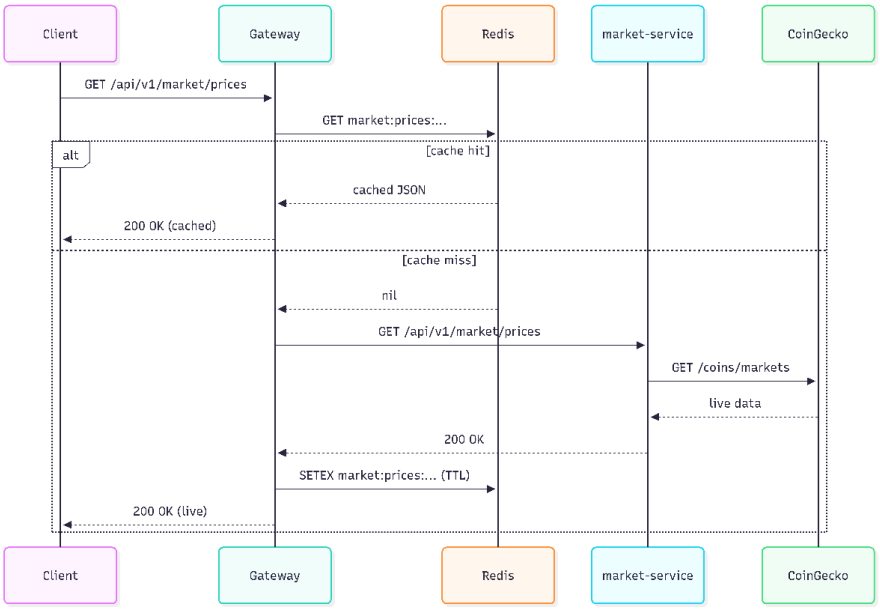


Figure 4: A cached vs. uncached `GET /api/v1/market/prices` request.

3.5 Behaviour

The gateway and both backend services are **stateless** between requests: any state they touch (the cache, the rate-limit counters, the databases) lives outside the process, in Redis or PostgreSQL, which is precisely what makes them safe to run as multiple replicas (section 3.8). The market-ingestion worker is the one genuinely **stateful** component: it holds an in-process polling loop with its own timing, and is not designed to be run as more than one replica; doing so would duplicate every fetch and every write, since nothing coordinates multiple ingestor instances against each other (this constraint is discussed further, as a known limitation, in section 9).

State updates therefore happen in exactly two places: the ingestion worker appends price snapshots on its fixed interval (FR7), and each backend service appends whatever it fetched immediately after answering a live (non-cached) request to it.

3.6 Data and Consistency Issues

3.6.1 CAP Theorem Position

The system deliberately prioritizes **Availability over Consistency** (AP), the opposite choice from a transactional system, and the correct one here: a client being served a price that is a few seconds stale is an acceptable, routine outcome; a client being refused an answer because the authoritative value could not be confirmed is not, for a read-only aggregator with no financial transactions riding on the exact figure. Framed against the CAP theorem [1, 4], under a network partition or a dependency outage the system

chooses to keep answering, whether from cache or by falling open past a failed rate-limit check, rather than to block or reject.

This AP choice is why both the gateway's cache and its distributed rate limiter are built to **fail open** (section 3.7) rather than fail closed, and why PostgreSQL replication (section 3.7.2) is asynchronous: a CP alternative was considered and rejected, since making every write wait for synchronous standby acknowledgement would trade away write availability for a consistency guarantee (zero data loss on failover) that nothing in section 2 actually demands.

3.6.2 Consistency Model

Both backend services persist every fetch to PostgreSQL, via the Exposed ORM over a HikariCP-managed connection pool, as **append-only** time series: a repeated observation of the same coin or the same article produces a *new* row rather than overwriting an existing one. This is a deliberate modelling decision (section 3.3), not an oversight: it means `getPriceHistory`/`getHistory` read a genuine time series rather than a single current value, which is exactly what FR2/FR4 need, at the cost of the tables only ever growing (mitigated by an index on `(coin_id, last_updated)` / `(category, published_at)` and a capped result size on every read). Reads of the cache, meanwhile, are only ever eventually consistent with the underlying database within the cache's short TTL, an explicit, accepted trade-off given the AP position above.

3.6.3 Shared Data and Synchronization

No domain data is shared *between* market-service and news-service: each owns its own database outright, and neither ever queries or writes to the other's. Because writes to each table are append-only, concurrent writers never contend on the same row; there is no update lock, no optimistic-concurrency version column, and no need for one, since nothing is ever updated in place. The one piece of state genuinely shared across component boundaries is the Redis cache and the Redis-backed rate-limit counters, shared across however many gateway (and, for the cache, news-service) replicas are running, which is the entire reason they live in Redis rather than in each process's own memory (section 3.8).

3.7 Fault-Tolerance

3.7.1 Service Isolation via Independent Databases

Because market-service and news-service are independent processes with independent databases and independent HikariCP connection pools, the failure or overload of one cannot directly cause the other to fail. A connection-pool saturation in news-service, for instance, does not affect market-service's pool.

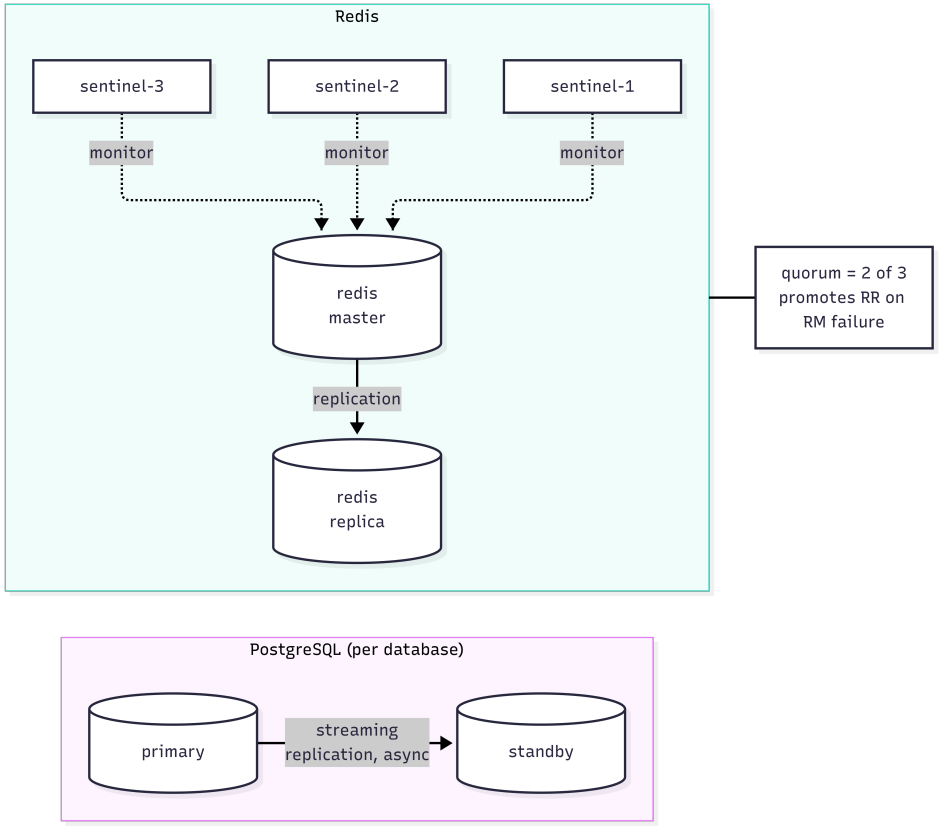


Figure 5: PostgreSQL streaming replication and the Redis Sentinel quorum.

3.7.2 Replication

Both PostgreSQL primaries (`postgres-market`, `postgres-news`) run with a dedicated streaming-replication standby [9], set up via a replication role and `pg_basebackup`. This exists purely for durability: losing a primary's container and volume would otherwise lose every price snapshot and news article ever collected, directly undermining FR2/FR4/NFR2. As discussed in section 3.6.1, replication here is asynchronous by deliberate choice; the practical consequence, honestly stated, is that a primary failing *at the exact moment of a write* could still lose that one write before it reached the standby, an accepted trade-off given this project's AP position, not an oversight. Redis similarly runs a primary/replica pair, monitored by a three-node Sentinel quorum (quorum size 2) that would promote the replica on a primary failure [7].

3.7.3 Fail-Open Cache and Rate Limiter

Both the gateway's proxy cache and its distributed rate limiter are designed to **fail open**: if a Redis/Sentinel read throws, the gateway logs a warning and falls through to the upstream fetch (for the cache) or allows the request through anyway (for the rate

limiter), rather than failing the whole request. This was not the original behaviour of the rate limiter: an internal audit of the system found that it originally had no such fallback and would have taken down *all* gateway traffic on a Redis blip; this was fixed before this report was written. Both backend services similarly catch and log (rather than propagate) a per-row persistence failure during a batch save, so one malformed row cannot take an entire successful fetch down with it.

3.7.4 Idempotent Backfill on Restart

As described in section 3.3.2, the `hasHistory` check that gates FR7's backfill is what makes `market-ingestor`'s crash recovery trivial: on restart, the worker simply re-runs its startup sequence against every top coin, and the gate itself determines which coins (if any) still need backfilling. No action log, checkpoint file, or separate "which coins are done" table is needed; the already-persisted data *is* the checkpoint.

3.7.5 Database Connection Retry and Startup Ordering

The market-ingestion worker retries its initial database connection with a capped linear backoff rather than crashing on a transient startup race against PostgreSQL still initializing. For this one component, failing to even start is worse than waiting, since the worker exists solely to write to the database. At the orchestration level, every Docker Compose service declares a `healthcheck`, and dependent services wait on `condition: service_healthy` rather than merely on the container having started, so a service never attempts to use a dependency that has launched but is not yet actually ready.

3.8 Availability

3.8.1 Caching

The gateway caches upstream responses (market prices, market history, news, news history) keyed by the request's distinguishing parameters, with a short TTL. When Redis is configured, this cache is shared across every gateway replica, which is what actually makes running more than one replica worthwhile, since otherwise each replica would cache (and warm its cache) independently. When Redis is not configured, the gateway falls back to a bounded in-process cache, which is correct for a single instance but would silently stop being a *shared* cache across multiple replicas, a limitation this project's design is explicit about rather than hiding.

3.8.2 Load Balancing

An nginx instance (`gateway-lb`) fronts however many `gateway` containers are running, forwarding to the `gateway` service name with DNS re-resolution on every request rather than once at startup, so that scaling the number of gateway replicas up or down is actually reflected. This is paired with the Redis-backed distributed rate limiter (section 3.7): load balancing across replicas would otherwise multiply the effective rate limit by the number of replicas, since Ktor's own built-in rate limiter keeps its counters in-process.

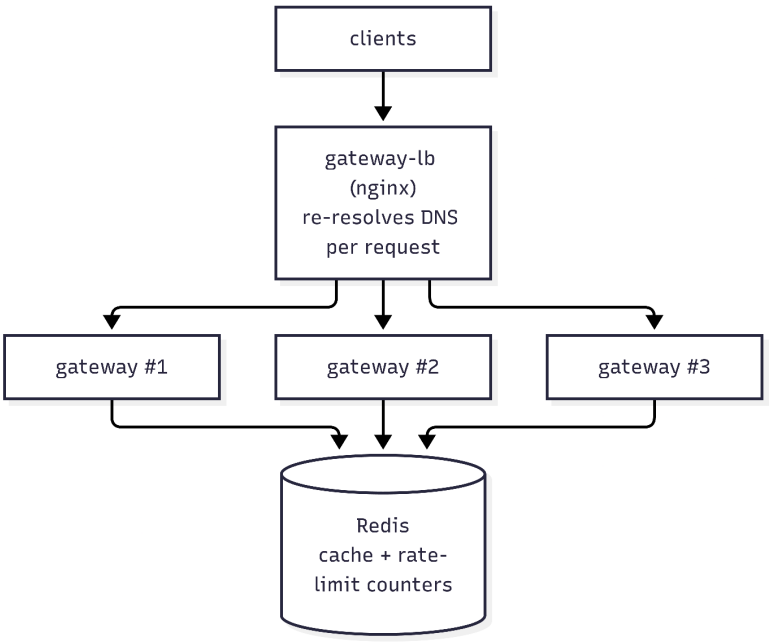


Figure 6: Gateway replicas behind `gateway-lb`, sharing state only via Redis.

3.8.3 Network Partition Behaviour

Two different partition scenarios are handled differently by design. A partition that isolates a PostgreSQL standby from its primary simply stalls replication until it heals, since the primary keeps serving reads and writes unaffected, given that replication here is asynchronous and one-way. A partition that isolates a minority of the Redis Sentinel quorum (at most one of three Sentinels, given quorum size 2) does not block failover decisions, since the remaining two Sentinels still form a quorum; a partition that isolated a *majority* of Sentinels would correctly block failover rather than risk a split-brain promotion, at the cost of not being able to fail over at all until it heals. That is a standard, intentional trade-off of quorum-based systems [7], not a flaw specific to this deployment.

3.9 Security

3.9.1 Current State

Orbital currently implements neither authentication nor authorization: every endpoint exposed by the gateway (`/api/v1/market/*`, `/api/v1/news/*`) is publicly readable, and no cryptographic token scheme protects any of them. This is a deliberate simplification given the product's current scope, a public, read-only market-data and news aggregator with no user accounts and no private data, but it does leave the security dimension of section 2.4 largely unaddressed by the implementation as it stands. The remaining subsections of this section describe a concrete design for closing that gap;

none of it has been implemented: no Keycloak container exists in the project's `docker-compose.yml`, the gateway installs no authentication plugin, and `orbital-app` has no login flow. It is presented as a proposal, and listed again as a concrete next step in section 9.

3.9.2 Proposed Identity Design (Not Implemented)

Rather than have each service manage credentials independently, Orbital would introduce a dedicated identity provider, Keycloak, as a new infrastructural component alongside the existing Postgres/Redis/Prometheus stack (section 3.2). Keycloak would own a single realm for the project and issue signed JSON Web Tokens (JWTs, [6]) after an OpenID Connect (OIDC) [8] Authorization Code flow with PKCE, the flow recommended for public clients that cannot keep a client secret, which matches `orbital-app`'s nature as a Kotlin Multiplatform client distributed as a desktop binary and a static web build, neither of which can safely embed one.

Centralizing identity this way is a natural fit for Orbital's existing architecture rather than a bolt-on. The client already talks exclusively to the gateway (section 3.1), so the gateway would remain the single trust boundary where tokens are validated, exactly as it is already the single point where caching and rate limiting are enforced. Validation itself would be stateless: the gateway would install an `Authentication` plugin with a JWT provider, fetching Keycloak's signing keys from its JWKS endpoint and caching them locally, refreshed on a TTL, the same pattern already used elsewhere in the system for distributed state (section 3.8). Because JWT validation needs no round-trip to Keycloak nor a shared session store, it composes cleanly with the horizontal-scalability work already done on the gateway: any number of gateway replicas behind the load balancer could validate tokens independently, with no new shared state and no new single point of failure introduced by the validation step itself.

3.9.3 Proposed Authorization Model (Not Implemented)

Authorization would be expressed through realm roles embedded as claims in the JWT (e.g. `user`, `premium`, `admin`), rather than through a separate access-control service. Given Orbital's current product shape, the most natural use of these roles would not be gatekeeping the existing read endpoints (which are deliberately public) but tiering them: authenticated requests could be granted a materially higher entry in the distributed rate limiter (section 3.8) than anonymous ones, and any future write-capable or administrative endpoint (e.g. forcing a cache invalidation, or re-triggering a historical backfill in the market ingestion worker) would require the `admin` role. This keeps the public-aggregator character of the product intact while still giving the system a real authorization story.

3.9.4 Keycloak's Own Fault Tolerance (Not Implemented)

Keycloak itself would need the same fault-tolerance treatment already applied to Orbital's other stateful components (section 3.7): it requires its own relational database for realm and user state, and running it as a single instance would reintroduce exactly

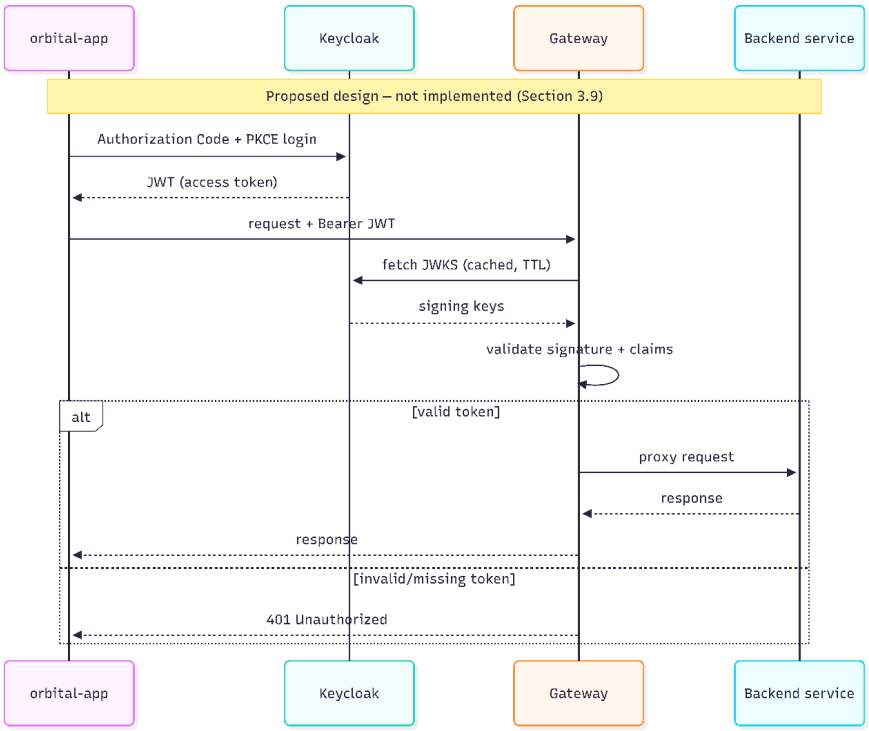


Figure 7: Proposed OIDC/JWT authentication flow (not implemented).

the kind of single point of failure that the Postgres and Redis replication work in this project was meant to eliminate. A production-oriented deployment would therefore either run Keycloak against one of the already-replicated Postgres instances, on a dedicated schema, or, more robustly, run Keycloak itself in clustered mode across two or more nodes.

4 Implementation

4.1 Network Protocols and Data Representation

Every component communicates over **HTTP/1.1**, carrying **JSON** bodies (via `kotlinx.serialization`): client to gateway, gateway to backend services, and backend services to CoinGecko. No binary or RPC protocol was introduced anywhere in the system; HTTP/JSON was sufficient for the synchronous request/reply pattern identified in section 3.4, and keeps every hop debuggable with a plain `curl`, which mattered in practice during this project's own manual verification work (section 5.2).

4.2 Data Persistence and Querying

Databases are queried with plain **SQL**, through Exposed's typed query DSL rather than raw query strings, which also means no part of this project builds a SQL string by hand from request input, closing off SQL injection as an attack surface by construction rather than by discipline. Schema is applied automatically at service startup via Exposed's `createMissingTablesAndColumns`, which creates missing tables, columns, and indexes, verified empirically in this project to also retroactively add a newly-introduced index to an already-existing table (section 5.1): the scenario an already-running deployment being upgraded would actually hit.

4.3 Authentication and Authorization

As detailed in section 3.9, no component is currently authenticated or authorized; there is no OAuth/JWT validation and no RBAC/ABAC check anywhere in the running system. A concrete, unimplemented design for closing this gap (OIDC via Keycloak, JWTs validated at the gateway, role claims driving a tiered rate limit) is proposed in section 3.9 and listed again in section 9.

4.4 Frontend Implementation

`orbital-app` is built with Compose Multiplatform, sharing one UI codebase across a `commonMain` source set and platform-specific `desktopMain`/`wasmJsMain` targets. State management uses Compose's own primitives directly (`remember`, `mutableStateOf`, and `LaunchedEffect`) rather than an external state-management library; for instance, a coin row's expanded/collapsed state and its lazily-fetched price history are both plain `remember`ed state, with `LaunchedEffect(coin.coinId)` triggering the history fetch only once a coin is actually expanded, and falling back to the live snapshot's short-range

`sparklineIn7d` data if the fetch returns fewer than two points (FR6). An injectable `fetchPriceHistory` function parameter is threaded down through the composable tree specifically so the chart-fetching behaviour can be swapped out or omitted (as the backend-free demo build does) without changing the UI components themselves. Network access goes through a thin `MarketApiClient` wrapping a Ktor HTTP client, calling the gateway's `/api/v1/market/prices` and `/api/v1/market/history/{coinId}` endpoints directly; the client never talks to market-service or news-service itself (section 3.1). Free-text coin search and news category filtering (FR5) are both plain client-side list filtering over whatever the gateway most recently returned, not separate backend endpoints.

4.5 Containerization and Orchestration

Every backend component is containerized with Docker. Each Ktor service (gateway, market-service, news-service) has a multi-stage `Dockerfile`: a build stage using `eclipse-temurin:21-jdk` to produce a fat JAR via Gradle's `shadowJar`, and a runtime stage using the much smaller `eclipse-temurin:21-jre`. `market-ingestor` reuses market-service's own build output via a separate `Dockerfile.ingestor` that differs only in its entrypoint (`com.orbital.market.MarketIngestorKt` instead of the HTTP server's `main`), rather than duplicating the build. One `docker-compose.yml` defines the entire system: all 17 services described in section 3.2, on one bridge network, with environment variables for service URLs, credentials, and feature toggles (such as `REDIS_SENTINEL_HOSTS`) injected per service, and health checks enforcing startup ordering (section 3.7.5). Section 6 gives the full per-service breakdown.

4.6 Technological Details

- **Kotlin** across the entire codebase.
- **Ktor** [5]: every backend HTTP server and client.
- **Compose Multiplatform**: the `orbital-app` client, JVM desktop and `wasmJs` web from one codebase.
- **Exposed** / **HikariCP** against **PostgreSQL 15** [9]: all persistence; **H2** (PostgreSQL-compatibility mode) stands in for it in the automated test suite.
- **Jedis**, behind an internal abstraction supporting either a plain `JedisPool` or a Sentinel-aware `JedisSentinelPool` [7].
- **nginx**: `gateway-lb`'s load balancer.
- **Prometheus** and **Grafana**: metrics collection and visualization, via a Micrometer-backed `/metrics` endpoint on each HTTP service.
- **Docker Compose** [3]: the sole deployment/orchestration mechanism.

- **Gradle** (multi-module), with **detekt** and **ktfmt/spotless** enforced in CI, and **GitHub Actions** + **release-please** automating testing, Docker image builds, and versioned releases from Conventional Commit history.

5 Validation

5.1 Automatic Testing

5.1.1 Test Infrastructure

Every JVM backend module has its own automated test suite, run with `./gradlew test` per module or `./gradlew check` for the full suite plus static analysis, executed on every push and pull request against `main`/`develop` by the `Test & Lint` GitHub Actions workflow. Persistence code runs against H2 in PostgreSQL-compatibility mode rather than a mock of the database, exercising the real Exposed queries and real schema. HTTP-level integration uses Ktor's `testApplication`, which runs the real routing and plugin pipeline (including CORS and rate limiting) against an in-process test server, with upstream HTTP dependencies replaced by Ktor's `MockEngine`. Pure logic with no external dependency, such as the distributed rate limiter's allow/reject/fail-open decision, is tested against a hand-written fake rather than a mocking framework, so the test has no dependency on a real Redis connection ever being reachable:

```
private class FakeRedisOps : RedisOps {
  private val counts = mutableMapOf<String, Long>()
  override fun incrWithExpire(key: String, windowSeconds: Int): Long =
      counts.merge(key, 1L, Long::plus) ?: 1L
}
```

5.1.2 Test Coverage

Tests are organized per module (20 for market-service, 14 for gateway, 10 for news-service, plus a handful across the two shared library modules) and cover the following requirement-grouped scenarios:

Market Data Ingestion and History (FR2, FR7). Asserts that `getPriceHistory` returns a coin's snapshots oldest-first, excluding other coins' snapshots; that `hasHistory` is false until a snapshot is stored for that coin and true afterward (the backfill gate from section 3.3.2); and, as a direct empirical answer to a question raised during this project's own audit, that `createMissingTablesAndColumns` actually adds the `(coin_id, last_updated)` index to a table created *without* it, simulating an already-running deployment being upgraded.

News Persistence and History (FR3, FR4). Asserts that `getHistory` returns the most recently published articles first, optionally filtered by category, and that the

`/api/v1/news/history` route returns stored articles filtered the same way at the HTTP level.

Gateway Caching and Rate Limiting (NFR1, NFR3, NFR4). Asserts that a repeated request for the same market prices within the cache TTL results in exactly one upstream call (NFR4); that a cache read failure falls through to the upstream fetch rather than failing the request (NFR1); that the distributed rate limiter allows calls up to its configured limit and rejects beyond it, tracking separate client keys independently (NFR3); and that it fails open when its backing store throws (NFR1, section 3.7.3), a regression test for a real bug found and fixed during this project's own internal audit. A further test asserts that a `429` response from the rate limiter still carries CORS headers, a regression test for an ordering bug found during the same audit.

Persistence Error Handling (NFR1-adjacent corner case). Asserts that a persistence write violating a column constraint is logged and skipped rather than crashing the surrounding batch, on both backend services.

5.1.3 How to Run

```
./gradlew test    # a single module's tests, e.g. ./gradlew :gateway:test
./gradlew check   # full suite, plus detekt and ktfmt/spotless
```

Test reports are available under each module's `build/reports/tests/` directory.

5.2 Acceptance Testing

Several properties of this system are either impractical or outright impossible to assert deterministically in a fast, isolated automated test, because they depend on real multi-process timing and real network behaviour: replication lag, Sentinel quorum formation and failover detection, and DNS-based load-balancer discovery. These were instead verified **manually** against the real `docker-compose` deployment:

- **PostgreSQL streaming replication** (NFR2): querying `pg_stat_replication` on the `postgres-market` primary showed an actively streaming replica (`state: streaming`) with matching sent/write/flush/replay log positions.
- **Redis Sentinel** (supporting NFR3's shared-cache precondition): a fatal configuration error was found and fixed first (Sentinel refuses to resolve a Docker service name as a monitor target unless `resolve-hostnames yes` is set), after which `sentinel master mymaster` against a live Sentinel instance reported the expected master, one discovered replica, and the other two Sentinels forming a three-node quorum.

Rationale for manual testing. Both scenarios require a live, multi-container environment with real inter-process timing (replication catching up, Sentinel's down-after-milliseconds detection window) that is impractical to assert deterministically in a fast unit or integration test without a full infrastructure-level testing harness. What was *not* verified manually, and is reported honestly as outstanding rather than assumed working, is discussed in section 9: the nginx load balancer's actual round-robin behaviour across multiple scaled `gateway` replicas, and the distributed rate limiter's behaviour under real concurrent multi-replica load.

6 Deployment

6.1 Prerequisites

The following software must be installed on the host machine:

- **Docker** with Docker Compose v2.
- **JDK 21** (for running the client/backend directly outside Docker; not required if only using `docker compose`).

6.2 Installation from Scratch

1. **Clone the repository and configure environment variables:**

   ```
   git clone <repository-url>
   cd orbital
   cp .env.example .env
   ```

   Every variable in `.env` has a safe, documented default for local development; only credential-related variables (`POSTGRES_MARKET_PASSWORD`, `POSTGRES_NEWS_PASSWORD`, `POSTGRES_REPLICATION_PASSWORD`, `GRAFANA_ADMIN_PASSWORD`, `PGADMIN_ADMIN_PASSWORD`) must be overridden before any deployment reachable over a real network. These replaced plaintext defaults that `docker-compose.yml` previously hardcoded directly, an issue identified and fixed during this project's own security audit.

2. **Build and start all services:**

   ```
   make up        # equivalent to: docker compose up --build -d
   ```

   On first startup, each Ktor service applies its schema automatically via Exposed (section 4.2) before accepting traffic.

3. **Verify deployment:**

```
curl localhost:8080/health
curl localhost:8080/api/v1/market/prices
curl localhost:8080/api/v1/news
```

Expected outcome: a healthy status from the first call, and live CoinGecko-sourced data from the second, within a few seconds.

6.2.1 Docker Compose Services

`docker-compose.yml` defines the following 17 services:

- **gateway:** The API gateway. Proxies, caches, and rate-limits requests to the two backend services. No fixed published port: `gateway-lb` is the host's actual entry point, so that it can be scaled to multiple replicas.
- **gateway-lb:** nginx, published on host port `8080`. Load-balances across however many `gateway` replicas are running.
- **market-service:** The market-data REST API, published on port `8081`. Depends on `postgres-market` and `redis`.
- **market-ingestor:** The standalone ingestion worker (FR7). No published port. Depends on `postgres-market`.
- **news-service:** The news REST API, published on port `8082`. Depends on `postgres-news` and the Redis/Sentinel cluster.
- **postgres-market / postgres-news:** The two independent PostgreSQL 15 primaries, published on ports `5432`/`5433`. Health-checked via `pg_isready` before dependent services start.
- **postgres-market-replica / postgres-news-replica:** Streaming-replication standbys of the two primaries, cloned via `pg_basebackup` on first start.
- **redis:** The Redis cache primary, published on port `6379`.
- **redis-replica:** A replica of `redis`, for Sentinel to promote on failover.
- **redis-sentinel-1/2/3:** The three-node Sentinel quorum (quorum size 2) monitoring `redis`/`redis-replica`.
- **prometheus / grafana:** Metrics collection (port `9090`) and visualization (port `3000`), with an auto-provisioned "Orbital Overview" dashboard.
- **pgadmin:** A PostgreSQL administration UI (port `5050`) for both databases.

All services share a single Docker bridge network (`orbital-net`), enabling inter-container communication by service name (section 3.2).

6.2.2 Startup Ordering and Health Checks

A critical aspect of the deployment is ensuring services start in the correct order. Each service that has one declares a `healthcheck`, and dependent services wait on `condition: service_healthy` rather than the weaker `condition: service_started` (section 3.7.5): the former waits for the dependency to genuinely be ready to accept connections, not merely for its container process to have launched. Without this, a service could start, attempt to connect to a database that has not finished initializing, and fail. `market-ingestor` specifically guards against this further with its own retry-with-backoff (section 3.7.5), since it is not gated by Compose health checks for its database dependency being fully migrated, only for PostgreSQL itself accepting connections.

6.3 Rebuilding Individual Services

A single service can be rebuilt and restarted without affecting the others:

```
docker compose up --build gateway
```

6.4 Schema Management

Unlike a Liquibase/Flyway-based migration tool, Orbital applies schema changes automatically at each service's startup, via Exposed's `createMissingTablesAndColumns` (section 4.2); there is no separate "run the migrations" command, and none is needed: starting (or restarting) `market-service` or `news-service` is itself the migration step. This was a deliberate simplicity trade-off for a project of this size, with the trade-off made explicit rather than assumed: section 5.1 reports an automated test that specifically verifies this mechanism retroactively adds a newly-introduced index to an already-existing table, since that guarantee cannot simply be assumed of every Exposed version.

7 User Guide

Two usage modes exist. The first is direct HTTP access, for scripting or inspection:

```
curl localhost:8080/api/v1/market/prices              # live market data
curl localhost:8080/api/v1/market/history/bitcoin      # a coin's ingested price history
curl localhost:8080/api/v1/news?category=crypto        # news, filtered by category
```

The second, and primary intended use, is `orbital-app`: a search box and list of cryptocurrencies with live price/change data; tapping a coin expands it to reveal a price-history chart (FR6), built from the backend's own ingested history once enough has accumulated, falling back to the live snapshot's short-range data otherwise; a separate tab lists news articles, filterable by category; both views auto-refresh periodically without user action. It is started with:

```
./gradlew :orbital-app:run                                # desktop window
./gradlew :orbital-app:wasmJsBrowserDevelopmentRun   # web, dev server + browser tab
```

A separate, backend-free build of the same client (market data only, fetched directly from CoinGecko, no gateway required) is deployed automatically on every release to `https://loch3f.github.io/orbital/`.

> *[Screenshots of the desktop and web client, showing the market list, an expanded coin's chart, and the news view, would be inserted here.]*

8 Self-evaluation

8.1 Aldo Visconti

This is a single-author project, so one section follows rather than one per group member.

Role. As the sole developer of this project, I was responsible for every aspect of the system: requirements elicitation, architectural design, backend implementation across the gateway and both backend services, the market-ingestion worker, the client application, the infrastructure work described in section 3.7 and section 3.8, and both automated and manual testing.

Strengths. The decision to decouple market-data ingestion from API traffic into a standalone worker (FR7, section 3.1) directly solved the problem that motivated it: a short, API-traffic-dependent price history. Its idempotent backfill gate (section 3.3.2) also turned out to double as the mechanism that makes the worker's crash recovery essentially free, without a separate checkpointing scheme. The fail-open design applied to the cache and, after a self-audit caught its absence, to the distributed rate limiter as well (section 3.7.3) proved to be the right default for a system whose CAP position is availability over consistency (section 3.6.1): it is a materially different, and better, failure mode than the one the rate limiter originally had. Verifying claims empirically rather than assuming them held up in practice: the Sentinel configuration had a real, fatal bug (`resolve-hostnames`) that only surfaced once I actually brought the quorum up, and the question of whether a schema migration tool would retroactively index an already-existing table turned out to be worth a real test rather than an assumption (section 5.1); in both cases, checking rather than assuming caught something that would otherwise have been a quiet, production-only surprise. Finally, keeping the backend split into independently buildable, CI-enforced Gradle modules from the start made it possible to add substantial new infrastructure (replication, Sentinel, a distributed rate limiter) across several iterations without destabilizing what already worked.

Weaknesses and Limitations. Working alone meant no second reviewer catching design blind spots as they happened; the internal audit process I ran against my own work, which is what actually found the rate limiter's fail-closed bug and the hardcoded

database credentials, was a partial substitute for that, but it is not equivalent to an independent reviewer, and I can't rule out that it missed things a second person would have caught. More concretely, several gaps are already honestly documented rather than glossed over: `orbital-app`, the one component real users actually touch, has no automated tests at all, unlike every backend module; the distributed rate limiter and load balancer were built specifically to make horizontal scaling safe but were never exercised against real concurrent multi-replica load, only against their own unit tests and a single-instance manual check; there is no authentication or authorization layer, only a proposed, unimplemented design (section 3.9); and PostgreSQL failover is manual in practice, since nothing promotes a standby automatically. Section 9 lists these and a few other concrete next steps in more detail.

9 Future works

Automatic database failover. Both PostgreSQL standbys (section 3.7) give Orbital a readable, up-to-date copy of each database, but nothing promotes a standby to primary or repoints a backend service's `DB_URL` automatically if a primary fails. That is normally the job of a dedicated tool such as Patroni or repmgr, deliberately left out of this project as a separate, larger piece of work rather than attempted partially.

Verifying scalability under real load. The nginx load balancer and the distributed rate limiter were both designed specifically to make running multiple `gateway` replicas safe (NFR3), and both compile and pass their own unit/integration tests, but neither has actually been exercised against several *scaled* replicas under concurrent load, e.g. `docker compose up --scale gateway=3` followed by sustained traffic, checking that the rate limit really is enforced once, globally, rather than once per replica. This is the natural next step beyond the manual verification already performed (section 5.2).

Synchronous replication. Section 3.6.1 discusses the AP position taken for both PostgreSQL pairs; a future iteration could make replication mode configurable per deployment, trading write latency for a zero-data-loss guarantee on failover where that trade is justified.

Automated testing of `orbital-app`. Every backend module has an automated test suite (section 5.1); the client application currently has none. This is a real gap specifically because the client is the one component actual users interact with directly.

Real news integration. News-service currently serves a mocked fallback feed rather than a paid NewsAPI subscription (section 2.4's economy discussion); wiring up the real integration behind the same interface it already exposes (`NewsApiClient`) would be a contained change once the cost trade-off is revisited.

Authentication and authorization. As discussed in section 3.9, Orbital has no authentication or authorization layer today. The concrete next step is the Keycloak-based design proposed there: a dedicated identity-provider service issuing OIDC/JWT tokens, validated statelessly at the gateway, with realm-role claims driving a tiered rate limit between authenticated and anonymous traffic, and gating any future write-capable endpoint. This remains future work rather than a completed feature.

References

[1] Eric A. Brewer. Towards robust distributed systems. In *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC)*, 2000.

[2] CoinGecko. CoinGecko API Documentation. `https://www.coingecko.com/en/api/documentation`, 2024. Accessed: 2026.

[3] Docker, Inc. Docker Compose Documentation. `https://docs.docker.com/compose/`, 2024. Accessed: 2026.

[4] Seth Gilbert and Nancy Lynch. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59, 2002.

[5] JetBrains s.r.o. Ktor Documentation. `https://ktor.io/docs/`, 2024. Accessed: 2026.

[6] Michael Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT) — rfc 7519. `https://datatracker.ietf.org/doc/html/rfc7519`, 2015.

[7] Redis, Inc. Redis Sentinel Documentation. `https://redis.io/docs/management/sentinel/`, 2024. Accessed: 2026.

[8] Nat Sakimura, John Bradley, Michael B. Jones, Breno de Medeiros, and Chuck Mortimore. OpenID Connect Core 1.0. `https://openid.net/specs/openid-connect-core-1_0.html`, 2014.

[9] The PostgreSQL Global Development Group. PostgreSQL 15 Documentation — Chapter 27: High Availability, Load Balancing, and Replication. `https://www.postgresql.org/docs/15/high-availability.html`, 2024. Accessed: 2026.